Planned OpenCTI Integration

Platform Development & Integration Documentation

Project: Python-Based Automated Web Penetration Testing Framework
Platform: OpenCTI (Open Cyber Threat Intelligence Platform)
Implementation status: Planned / Partially prepared
Deployment approach: Local Docker-based educational environment

---

1. Introduction

OpenCTI (Open Cyber Threat Intelligence Platform) is a platform designed to collect, organize, connect, and visualize cybersecurity threat intelligence.

The purpose of incorporating OpenCTI into this project is to extend the capabilities of the Python-based automated web penetration-testing framework by introducing a centralized threat intelligence component.

The existing framework focuses on identifying potential security weaknesses in web applications, validating findings, collecting supporting evidence, and generating security reports. OpenCTI would provide an additional layer for organizing relevant threat intelligence and understanding how identified security issues relate to known vulnerabilities, adversary techniques, and attack patterns.

OpenCTI uses the STIX 2.1 threat intelligence standard to structure information and provides a GraphQL API for integration with external applications. OpenCTI documentation.

2. How OpenCTI Benefits the Project

The integration is intended to improve the overall usefulness of the penetration-testing framework without replacing its existing functionality.

2.1 Centralized Threat Intelligence

OpenCTI can provide a centralized location for collecting and organizing information from threat intelligence sources.

Potential information includes:

- Known vulnerabilities and associated references.
- MITRE ATT&CK tactics and techniques.
- Malware and adversary information.
- Indicators of compromise (IOCs).
- Relevant threat intelligence reports.
- Relationships between threat actors, tools, and attack techniques.

This would allow the project to incorporate external cybersecurity knowledge alongside its own testing results.

2.2 Improved Context for Security Findings

The existing framework identifies potential vulnerabilities through automated checks, including SQL injection indicators, external redirects, and missing security headers.

OpenCTI could help provide additional context around these findings.

For example:

- A SQL injection finding could be associated with relevant vulnerability intelligence and references.
- An identified web application endpoint could be reviewed alongside relevant threat intelligence.
- MITRE ATT&CK techniques could provide context about how attackers commonly approach web applications.
- Relevant threat intelligence could help explain why a particular security weakness matters.

Important: OpenCTI would not independently validate vulnerabilities detected by the framework. The framework's existing validation and confidence mechanisms would remain responsible for determining the status of its findings.

2.3 Improved Reporting

The existing framework generates HTML security reports containing findings, severity levels, reproduction information, remediation guidance, and supporting evidence.

A future integration could add relevant threat intelligence references to these reports.

Potential additions include:

- Related MITRE ATT&CK techniques.
- Relevant vulnerability references.
- Associated threat intelligence.
- External intelligence source references.
- Additional context about the potential impact of a finding.

This would make the reports more informative and help connect technical findings with broader cybersecurity threats.

2.4 Future Integration and Expandability

OpenCTI provides an API and connector architecture that allows external applications to exchange information with the platform.

This creates an opportunity to develop a custom Python integration that transfers selected findings from the penetration-testing framework into OpenCTI.

OpenCTI supports importing external intelligence, enriching existing information, importing files, exporting information, and consuming data streams through connectors. OpenCTI connector documentation.

---

3. Proposed Integration Architecture

The proposed architecture keeps the penetration-testing framework as the primary scanning and evidence-collection application.

OpenCTI operates as a separate threat intelligence platform.

Proposed data flow

1. The penetration-testing framework scans an authorized target.
2. The framework identifies potential security findings.
3. Findings are validated and assigned severity, confidence, and status.
4. Evidence is collected and preserved using the existing evidence-management system.
5. Selected findings and relevant metadata are prepared for integration.
6. A future integration component converts compatible information into an appropriate STIX 2.1 representation.
7. The information is submitted to OpenCTI through its API or a custom connector.
8. OpenCTI organizes the imported information and relates it to relevant threat intelligence.
9. Relevant intelligence can be reviewed and incorporated into future security reports.

Architecture overview

```
None
       Authorized Web Application
                  |
                  v
```



This is a proposed architecture. The integration component and automated data transfer have not yet been implemented.

---

4. Planned Implementation Steps

Implementation will be divided into stages so that OpenCTI does not interfere with the development and testing of the main penetration-testing framework.

Stage 1: Complete the Core Penetration-Testing Framework

Priority: High

Before implementing OpenCTI integration, the existing framework should have a stable foundation.

Tasks:

1. Complete the automated scanning workflow.
2. Verify the vulnerability detection checks.
3. Confirm finding validation and confidence assignments.
4. Complete evidence collection and SHA-256 integrity verification.
5. Confirm that evidence identifiers correctly link to findings.
6. Finalize HTML security report generation.
7. Verify that the framework can operate independently.

Expected result: A stable penetration-testing framework that can generate structured findings and reports without relying on OpenCTI.

Stage 2: Prepare the OpenCTI Environment

Priority: Deferred

OpenCTI will be deployed as a separate application using Docker Compose.

The planned environment includes:

Component	Purpose
OpenCTI Platform	Main web application and API
OpenSearch	Search and indexing
Redis	Caching and application support
RabbitMQ	Message processing

Silo	S3-compatible object storage
OpenCTI Worker	Asynchronous processing
MITRE ATT&CK Connector	Importing MITRE intelligence

The initial deployment uses a reduced educational configuration rather than the full production deployment.

Implementation tasks:

1. Install and configure Docker Desktop.
2. Clone the official OpenCTI Docker repository.
3. Configure environment variables and credentials.
4. Configure the required supporting services.
5. Start and verify the supporting containers.
6. Start the OpenCTI platform.
7. Access the web interface locally.
8. Verify administrator access and application health.

The official OpenCTI documentation provides Docker deployment instructions and explains the platform's infrastructure requirements. Installation documentation.

Stage 3: Configure Threat Intelligence Feeds

Priority: Deferred

Once OpenCTI is operational, the next step will be configuring a threat intelligence source.

The initial planned source is MITRE ATT&CK.

Implementation tasks:

1. Configure the MITRE ATT&CK connector.
2. Create an appropriate connector identity and access token.
3. Configure the connector's connection to OpenCTI.
4. Start the connector.
5. Allow the connector to retrieve and import the available intelligence.
6. Verify that imported information appears in OpenCTI.
7. Review imported tactics, techniques, and related information.
8. Capture evidence demonstrating successful data ingestion.

The MITRE ATT&CK connector is an external-import connector. It imports structured threat intelligence into OpenCTI rather than scanning the user's web application.

OpenCTI connector configuration documentation.

Stage 4: Develop the Python Integration

Priority: Future development

After the main framework and OpenCTI platform are stable, a separate integration module can be developed.

Proposed module:

`integrations/opencti_integration.py`

Potential responsibilities:

- Establish a connection to the OpenCTI API.
- Authenticate using a dedicated integration token.
- Read selected findings from the framework.
- Normalize finding information.
- Convert supported information into STIX 2.1 objects.
- Submit information to OpenCTI.
- Record successful and unsuccessful transfers.
- Avoid duplicate submissions.

The integration should use a dedicated OpenCTI service account rather than reusing the administrator account.

Potential data mapping:

Existing framework field	Proposed integration use
Finding ID	Preserve the original finding reference
Finding title	Descriptive title
Severity	Finding severity metadata
Status	Validation status
Confidence	Confidence information
Target URL	Relevant URL observable, where appropriate
Endpoint	Relevant endpoint information
Description	Finding details
Evidence ID	Reference to preserved evidence

SHA-256	Evidence integrity reference
References	Related vulnerability or intelligence references

Not every framework field should automatically become a STIX object. The integration must distinguish between vulnerabilities, observable data, indicators, reports, and other intelligence concepts.

For example, a URL discovered during testing is not automatically a malicious indicator. Likewise, a potential SQL injection finding should not automatically be represented as a confirmed vulnerability.

The integration should preserve the framework's original finding status and confidence rather than treating imported information as independently verified.

OpenCTI's connector development documentation describes its Python SDK, connector types, and STIX 2.1 data exchange. Connector development.

Stage 5: Integrate Threat Intelligence into Reporting

Priority: Future development

Once data transfer is functioning, the reporting system can be extended.

Potential changes to `reports/html_report.py` include:

- Displaying relevant threat intelligence references.
- Displaying related ATT&CK techniques when supported by evidence.
- Including OpenCTI object references.
- Displaying the status of intelligence enrichment.
- Recording whether a finding was successfully transferred.

The original vulnerability finding and its evidence must remain available even if OpenCTI is unavailable.

Stage 6: Testing and Verification

Priority: Future development

The completed integration should be tested in the isolated lab environment.

Planned tests:

1. Verify successful API authentication.
2. Submit a test finding.
3. Confirm that the information appears in OpenCTI.

4. Verify that the original finding ID is preserved.
5. Confirm that relevant metadata is retained.
6. Test duplicate submission handling.
7. Test API failure handling.
8. Confirm that the original evidence files and hashes remain unchanged.
9. Verify that the integration does not prevent normal scanning or report generation.

---

5. Security and Configuration Considerations

The integration will require careful handling of credentials, evidence, and network access.

Planned security measures include:

- Store OpenCTI credentials in environment variables rather than source code.
- Use dedicated service accounts and tokens.
- Restrict access to the local OpenCTI web interface.
- Avoid exposing OpenSearch or other internal services to external networks.
- Preserve evidence files in the existing evidence directory.
- Do not transfer unnecessary sensitive information into OpenCTI.
- Maintain original evidence identifiers and SHA-256 hashes.
- Log integration activity and errors.
- Use authorized lab targets and approved intelligence sources.

The initial local deployment is intended for educational testing, not production use.

The available Mac environment has limited Docker memory compared with a typical production OpenCTI deployment. Therefore, the reduced configuration is intended to support experimentation and demonstrate the integration concept, not to provide production-level capacity.

---

6. Deployment Documentation and Evidence Requirements

When implementation resumes, the following evidence should be collected for the Platform Development & Integration rubric.

Rubric requirement	Planned evidence
Docker configuration	Docker Compose configuration and container status

Platform installation	OpenCTI web interface and successful login
Feed integration	MITRE ATT&CK connector configuration
Data flow	Connector logs and imported intelligence records
Security configuration	Environment configuration with secrets redacted
Analytics dashboards	OpenCTI dashboard screenshots and relevant metrics
Integration	Python integration module and successful API transfer
Deployment documentation	Architecture diagram, setup instructions, and operational procedures

Evidence should include timestamps and descriptions of the actions performed.

Configuration files may be included in the project documentation, but passwords, API tokens, and other secrets must be redacted.

---

7. Current Implementation Status

As of November 5, 2026:

Task	Status
Select OpenCTI as the threat intelligence platform	Completed
Review OpenCTI architecture and dependencies	Completed
Confirm ARM64 image compatibility	Completed
Clone the official Docker repository	Completed
Create a reduced Docker Compose configuration	Completed
Generate local environment credentials	Completed
Download required Docker images	Completed
Start Redis	Completed

Start RabbitMQ	Completed
Start Silo object storage	Completed
Start OpenSearch	Started successfully; final health verification pending
Start the OpenCTI platform	Deferred
Configure MITRE ATT&CK feed	Deferred
Develop Python integration	Not started
Integrate intelligence into security reports	Not started
Complete integration testing	Not started

Important implementation note: The supporting infrastructure has been started locally, but this does not mean the OpenCTI platform has been fully deployed or that threat intelligence is being imported. The remaining deployment and integration steps are planned work.

---

8. Recommended Development Order

OpenCTI should be treated as an extension of the main project rather than a prerequisite for completing the penetration-testing framework.

The recommended order is:

1. Complete and test the Python penetration-testing framework.
2. Finalize evidence management and chain-of-custody functionality.
3. Complete the HTML reporting system.
4. Finish the remaining required security testing and forensic documentation.
5. Return to the OpenCTI deployment.
6. Configure MITRE ATT&CK intelligence ingestion.
7. Develop the Python-to-OpenCTI integration.
8. Add threat intelligence references to the final security report.
9. Test and document the complete integration.

This approach reduces the risk of spending too much development time on infrastructure before the primary application is complete.

---

9. Conclusion

OpenCTI is a planned enhancement to the Python-based Automated Web Penetration Testing Framework.

Its purpose is to introduce threat intelligence management, external intelligence feeds, and the possibility of connecting discovered security findings with broader cybersecurity knowledge.

The proposed integration would allow the framework to move beyond identifying potential web application weaknesses and eventually provide additional context about relevant vulnerabilities, indicators, and adversary techniques.

The existing penetration-testing framework will remain responsible for scanning, finding validation, evidence collection, and security report generation. OpenCTI will serve as a complementary threat intelligence platform.

Implementation has been intentionally deferred until the core framework and its required evidence and reporting components are stable.

Final project objective: Develop a modular penetration-testing framework that can operate independently while supporting future integration with a centralized threat intelligence platform.